

eSim Simulation Bridge

KiCad Plugin Suite for eSim/ngspice Simulation

Design Document

Plugin Name: eSim Simulation Bridge v1.0.0

Author: Imran Farhat

Institution: VIT Bhopal University

Submitted For: FOSSEE SLI Spring 2026 - KiCad Plugin Development

Mentor: Sumanto Kar, IIT Bombay

Principal Investigator: Prof. Prabhu Ramachandran, IIT Bombay

KiCad Version: 8.0

Platform: Ubuntu Linux 24.04 LTS (VirtualBox)

Environment: KiCad 8.0 + eSim 2.5 + ngspice 42

License: GPL-3.0

GitHub: [ImranFarhat eSim Simulation Bridge](#)

May 2026

TABLE OF CONTENT

1. Problem Statement.....	3
2. Solution Overview.....	3
3. File Structure.....	4
4. Architecture Overview.....	4
5. Module Breakdown.....	6
5.1 ESimBridgePlugin - Main Plugin Class (esim_bridge.py).....	6
5.2 KicadToNgspiceDialog - 6-Tab Configuration Dialog.....	7
Tab 1 - Analysis.....	7
Tab 2 - Source Details.....	8
Tab 3 - Ngspice Model.....	8
Tab 4 - Device Modeling.....	8
Tab 5 - Subcircuits.....	8
Tab 6 - Microcontroller.....	8
5.3 SPICEConverter.....	8
Net Name Cleaning Rules.....	9
5.4 SPICEModelLibrary.....	9
5.5 ExternalModelLoader.....	10
5.6 PreflightChecker.....	10
Environment Checks (run_all_checks).....	11
Netlist Checks (check_netlist).....	11
5.7 NgspiceWaveformViewer.....	11
5.8 NgspiceRawParser.....	13
5.9 SimulationReadyDialog.....	13
5.10 ESimLauncher.....	14
6. SPICE Model Auto-Linker (esim_spice_linker.py, v1.0.0).....	15
6.1 Overview.....	15
6.2 ESimLibraryScanner.....	15
Match Scoring Algorithm.....	15
6.3 ModelMatcher - 5-Tier Search Cascade.....	16
6.4 TextbookModelGenerator.....	17
6.5 ModelStatusReport Dialog.....	17
7. Python Plot Window Integration (ngspiceSimulation package).....	18
7.1 Overview.....	18
7.2 Three-Panel Layout.....	19
7.3 Analysis-Specific Rendering.....	19
7.4 Digital Timing Diagram.....	20
7.5 Cursor Measurements and Multimeter.....	20
7.6 Appconfig Compatibility Layer.....	21
8. Validated Test Circuits.....	21
8.1 Circuit 1: Three-Phase Full-Wave Diode Bridge Rectifier with LC Filter.....	21
8.2 Circuit 2: Two-Stage Stagger-Tuned Amplifier.....	23
9. Data Flow.....	26

10. Known Limitations and Scope.....	27
10.1 ngspice-Simulatable Circuits Only.....	27
10.2 Standard Digital ICs.....	27
10.3 UTF-8 Cosmetic Popup.....	27
10.4 Parametric Sweep Leftover.....	27
10.5 ngspice 42 Pole-Zero Bug.....	27
11. Technical Decisions and Bugs Resolved.....	27
12. Extensibility.....	28
13. Conclusion.....	29
14. Future Work.....	30

1. Problem Statement

KiCad and eSim are both free, open-source EDA tools widely used in Indian academic institutions. KiCad handles schematic capture and PCB design; eSim (developed by FOSSEE, IIT Bombay) provides SPICE-based circuit simulation via ngspice.

The workflow to simulate a KiCad schematic in eSim is entirely manual:

1. Export netlist from KiCad (File → Export → Netlist)
2. Open eSim and create a new project with the correct name
3. Import the netlist file into the eSim project folder
4. For every active component: open Model Editor → search for SPICE model → download → link → save
5. Configure simulation type and parameters (AC/DC/Transient/OP etc.)
6. Run ngspice and open eSim plotter to select signals to visualise

This process takes 10-15 minutes per schematic and must be repeated from scratch whenever a component value changes. eSim's built-in waveform plotter also lacks FFT spectra, Bode plots, parametric sweeps, and interactive cursor readout.

Goal: Reduce this multi-step manual workflow to a single toolbar button click inside KiCad, and add professional waveform analysis capabilities beyond eSim's built-in plotter.

2. Solution Overview

The solution is a single KiCad action plugin - the eSim Simulation Bridge - installed as a single package with three source files:

File	Role
esim_bridge.py (v1.0.0)	Core plugin: netlist export, SPICE conversion, 6-tab dialog, 7 analysis types, ngspice execution, embedded waveform viewer, eSim project generation, eSim launch
esim_spice_linker.py (v1.0.0)	SPICE Model Auto-Linker: scans eSim's open-source library (647 files), automatically resolves and injects SPICE models for every active component
ngspiceSimulation/ package	Python plot window (plot_window.py, data_extraction.py, plotting_widgets.py): full PyQt5 waveform viewer integrated dynamically via importlib

Together they reduce the workflow to 5 steps in approximately 30 seconds:

1. Open KiCad → draw schematic → save (Ctrl+S)
2. Switch to PCB Editor → click the eSim Simulation Bridge toolbar icon
3. Select analysis type and parameters in the 6-tab dialog → eSim-SPICE shows Model Coverage Report
4. Click Convert → review SPICE preview → click Run with ngspice
5. Interact with the waveform viewer: FFT, Bode plot, cursor, parametric sweep

A key design principle throughout the plugin is accuracy over convenience: every numerical value displayed in the stats panel is read directly from ngspice's binary .raw output at

runtime. No hardcoded circuit-specific defaults are used anywhere. This ensures that results are always derived from actual simulation data, regardless of the circuit being analysed.

A second principle is transparency: the plugin mirrors eSim's own interface wherever possible. The 6-tab dialog replicates eSim's KicadToNgspice window so that students already familiar with eSim feel at home immediately. The SPICE file generated is human-readable and shown to the user before simulation begins.

3. File Structure

Repository structure under ImranFarhat_eSim_Simulation_Bridge/:

ImranFarhat_eSim_Simulation_Bridge/

```

├── eSim_Simulation_Bridge/      ← install this folder to KiCad scripting/plugins/
│   ├── __init__.py             ← package entry point
│   ├── esim_bridge.py          ← core plugin v1.0.0
│   ├── esim_spice_linker.py     ← SPICE Model Auto-Linker v1.0.0
│   ├── icon.png                ← KiCad toolbar icon
│   └── configuration/
│       ├── __init__.py
│       └── AppConfig.py        ← eSim compatibility layer
├── ngspiceSimulation/
│   ├── __init__.py
│   ├── plot_window.py          ← PyQt5 main viewer
│   ├── data_extraction.py      ← ngspice output parser
│   └── plotting_widgets.py     ← UI widgets
├── README.md                   ← overview, installation, features
├── INSTALLATION_GUIDE.md       ← step-by-step installation
├── QUICK_REFERENCE.md          ← workflow cheat sheet
├── KNOWN_LIMITATIONS.md        ← limitations reference
├── DESIGN_DOCUMENT.pdf         ← this document
└── metadata.json               ← KiCad Plugin Manager metadata

```

4. Architecture Overview

The plugin uses a sequential pipeline architecture. Each stage transforms the data and passes it to the next stage with no shared global state between them. This design was chosen deliberately: each stage can be tested independently, failures are localised, and the pipeline can be extended by inserting new stages without modifying existing ones.

The pipeline separates concerns cleanly: schematic-side operations (netlist export, parsing, dialog, preflight) are handled entirely by `esim_bridge.py` before any SPICE-specific logic begins. SPICE model resolution is handled by a separate module (`esim_spice_linker.py`) that is imported lazily only when needed. Visualisation is handled by two independent viewers - the embedded wxPython viewer for immediate results and the PyQt5 plot window for eSim-compatible analysis.

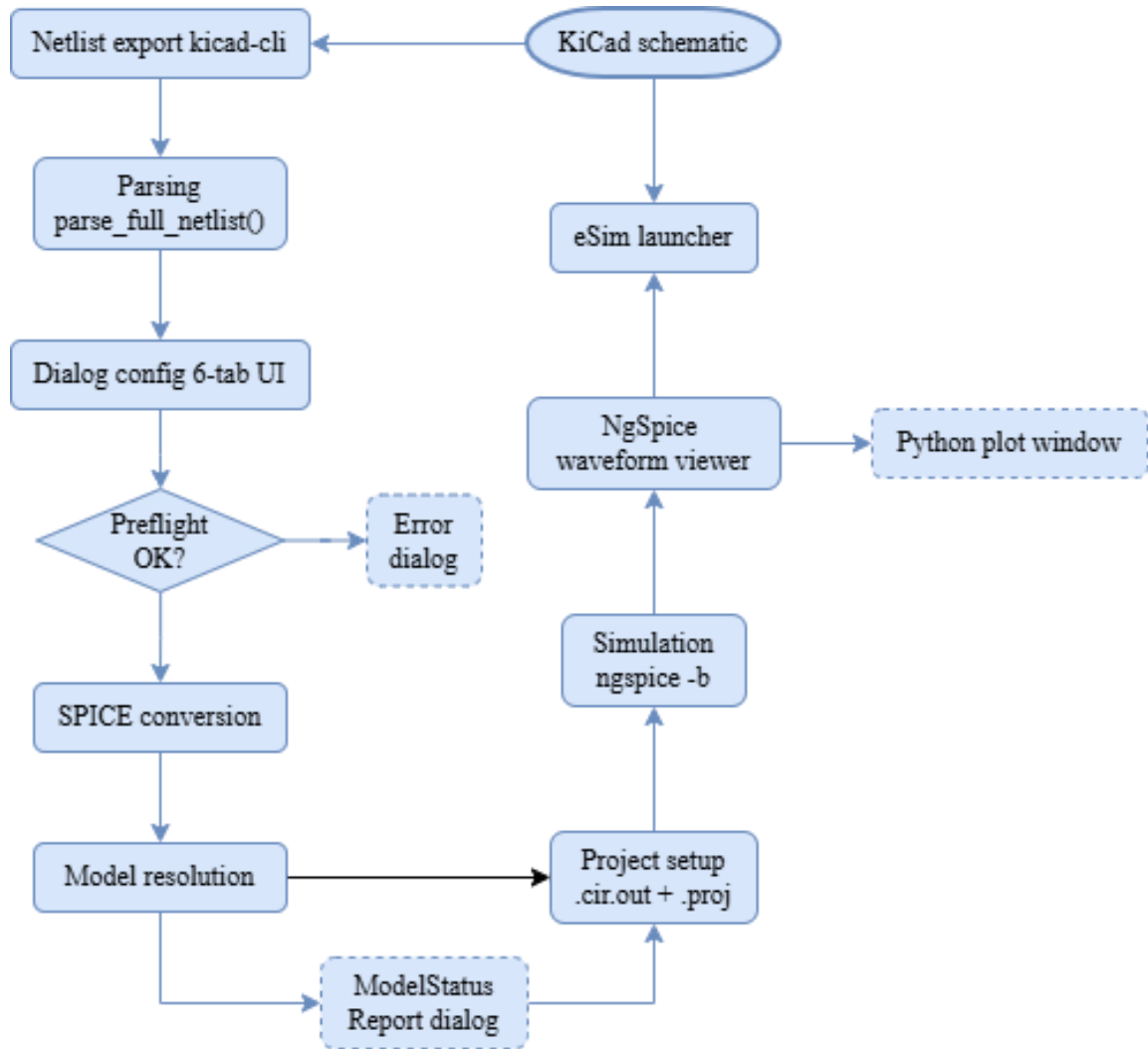


Figure 4.1: End-to-end pipeline of the eSim Simulation Bridge plugin - from KiCad schematic capture through netlist export, SPICE conversion, model resolution, ngspice batch simulation, and interactive waveform visualisation.

Stage	Input	Output	Component
1. Netlist Export	.kicad_sch	.net (KiCad s-expression)	kicad-cli subprocess
2. Parsing	.net file	components + nets dicts	SPICEConverter.parse_full_netlist()
3. Dialog Config	components dict	analysis type + params	KicadToNgspiceDialog (6 tabs)
3b. Preflight	components + nets	error / warning list	PreflightChecker.check_netlist()
4. SPICE Conversion	components + models	.cir file	SPICEConverter.convert()

Stage	Input	Output	Component
5. Model Resolution	.cir + match results	.cir with all models	SPICEAutoLinker
6. Project Setup	Final .cir	.cir.out + .proj + analysis	ESimBridgePlugin.Run()
7. Simulation	.cir.out	.raw waveform file	ngspice -b (batch mode)
8. Visualisation	.raw file	Interactive waveform	NgspiceWaveformViewer
9. eSim Launch	Project folder	eSim running	ESimLauncher.launch()

5. Module Breakdown

5.1 ESimBridgePlugin - Main Plugin Class (esim_bridge.py)

ESimBridgePlugin is the orchestrator of the entire pipeline. It inherits from pcbnew.ActionPlugin and is registered with KiCad at module load time via ESimBridgePlugin().register(). When the user clicks the toolbar button, KiCad calls Run() - the single method that drives every stage from netlist export to simulation result display.

The Run() method is designed to fail gracefully at each stage: if the schematic is not found, it shows an error and stops; if preflight checks fail, it shows specific error messages with actionable suggestions; if ngspice produces no output, it reports the failure rather than crashing. All events are logged with timestamps to ~/.local/share/kicad/esim_bridge.log for post-mortem debugging.

Method	Purpose
defaults()	Register plugin name, icon, description, and toolbar button with KiCad
Run()	Main orchestration - calls all pipeline stages in sequence with full error handling
get_schematic_path()	Resolves .kicad_sch from the open PCB project; falls back to wx.FileDialog
export_netlist()	Shells out to kicad-cli sch export netlist --format kicadsexpr



Figure 5.1: The eSim Simulation Bridge toolbar button visible in the KiCad 8.0 PCB Editor top toolbar. Clicking this button triggers the complete simulation pipeline.

5.2 KicadToNgspiceDialog - 6-Tab Configuration Dialog

The KicadToNgspiceDialog is the primary user interface of the plugin. It mirrors eSim's own KicadToNgspice converter window so that students already familiar with eSim can use the plugin without relearning a new interface. The dialog is built dynamically after netlist parsing - each tab is populated with the actual components found in the schematic, not generic placeholders.

A key implementation decision was to use wx.Panel wrappers for each analysis parameter group rather than wx.StaticBoxSizer. On GTK/Linux, StaticBoxSizer.Show(False) does not release vertical space, leaving large gaps in the layout when analysis types are switched. wx.Panel correctly collapses the hidden space. All parameter values are persisted to an XML file and restored on the next session, so users do not need to re-enter their simulation settings every time.

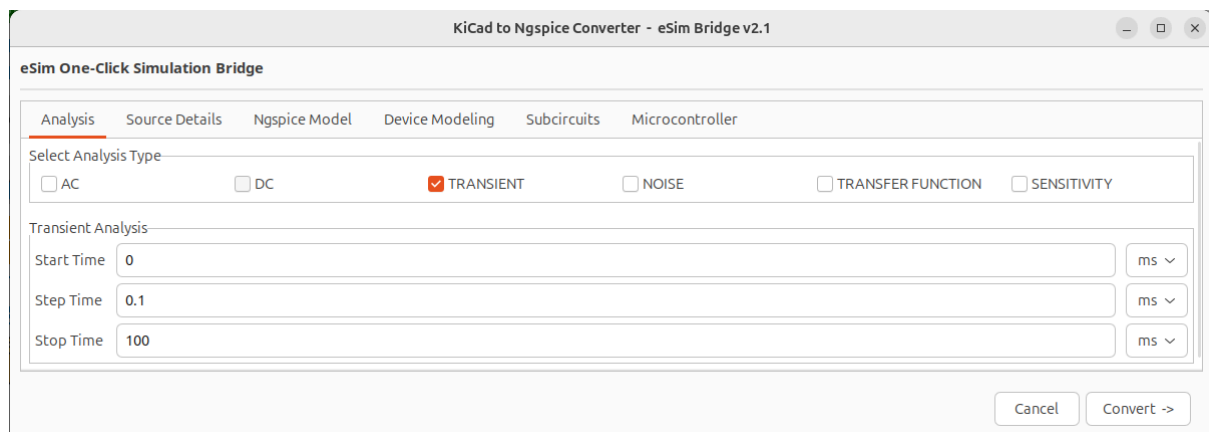


Figure 5.2: The KicadToNgspiceDialog showing the Analysis tab with Transient analysis selected. The Start Time, Step Time, and Stop Time parameter fields are visible. The other six analysis types are shown as unselected checkboxes.

Tab 1 - Analysis

Seven analysis types as checkboxes with radio-group behaviour (only one active at a time). Each type has its own wx.Panel that shows/hides. All values are saved to ~/.esim-bridge/KicadToNgspice_Previous_Values.xml and restored on next session.

Analysis Type	Key Parameters	Result
Transient	Start Time, Step Time, Stop Time	Waveform Viewer
AC Sweep	Scale (Lin/Dec/Oct), Start Freq, Stop Freq, Points	Waveform Viewer
DC Sweep	Source 1, Start, Increment, Stop (+ optional Source 2)	Waveform Viewer
Operating Point	(none)	MessageBox popup
Noise	Output Node, Input Source, Freq Range	MessageBox popup
Transfer Function	Output Node, Input Source	MessageBox popup
Sensitivity	Output Variable (e.g. V(vmid))	MessageBox popup

Tab 2 - Source Details

Dynamically built from all V/I prefix components found in the parsed netlist. Each source shows a type dropdown (dc/ac/sine/pulse/pwl/exp) with matching parameter fields. Pre-populated from KiCad's Sim.Type and Sim.Params properties.

Tab 3 - Ngspice Model

For U-prefix components. Reads eSim's modelParamXML/ directory and renders parameter entry fields from the XML schema. Mirrors eSim's own Model.py tab exactly.

Tab 4 - Device Modeling

For Q/D/J/M/S prefix components. File picker for external .lib files. MOSFET components additionally expose Width, Length, and Multiplication Factor fields.

Tab 5 - Subcircuits

For X-prefix components. Directory picker for subcircuit folder. Port count validation is performed against the component's actual pin count before the selection is accepted.

Tab 6 - Microcontroller

Detects MCU-type components (ATtiny, Arduino, ATmega, PIC, STM32, ESP) by value keyword. Checks whether NGHDL is installed, displays its home path if found, and provides a hex file picker for loading compiled firmware.

5.3 SPICEConverter

The SPICEConverter is responsible for transforming the parsed KiCad netlist into a syntactically valid SPICE deck. This is the most complex component in the plugin because it must handle the impedance mismatch between KiCad's component model (which uses schematic symbols with arbitrary value fields) and ngspice's strict SPICE syntax requirements.

The converter uses a priority-ordered prefix dispatcher: multi-character prefixes (MK, BT, SW) are checked before single-character prefixes to prevent false matches. For example, without this ordering, a microphone component MK1 would be incorrectly dispatched to the MOSFET handler because Python's str.startswith('M') would match both M and MK. Net names are sanitised to remove characters illegal in SPICE node names, with GND mapped to node 0 and names truncated to 20 characters.

Prefix	Component	SPICE Handling
MK*	Microphone	10mV AC source at 1kHz - VMK1 <non-gnd-node> 0 AC 0.01 SIN(0 0.01 1k)
BT*	Battery	DC voltage source - VBT1 node+ node- DC <value>
SW*	Switch	1 Ω resistor (closed state) - RSW1 node+ node- 1
R	Resistor	Direct value with regex sanitisation (strips text after first space; fallback to 1k)
C	Capacitor	Direct value

Prefix	Component	SPICE Handling
L	Inductor	Direct value
V	Voltage source	Full Sim.Type + Sim.Params handling: SIN, PULSE, DC, AC, EXP, PWL
I	Current source	DC value
D	Diode / LED	Auto-model from ExternalModelLoader → SPICEModelLibrary → eSim-SPICE
Q	BJT Transistor	Auto-model NPN/PNP; KiCad pin 1=Collector, pin 2=Base, pin 3=Emitter
M	MOSFET	Auto-model NMOS/PMOS; KiCad pin 1=Gate, pin 2=Drain, pin 3=Source
J	JFET	Auto-model NJF/PJF
U / X	IC / Subcircuit	5-step lookup cascade → built-in → external → eSim-SPICE
S	Voltage-controlled switch	1Ω resistor
F	Fuse	0.01Ω resistor
T	Transformer	Unsupported comment - manual .subckt required

Net Name Cleaning Rules

- GND / GROUND / VSS / 0 → SPICE node 0
- Strip leading /
- Replace non-alphanumeric with _, collapse multiple underscores
- Truncate to 20 characters
- Prepend N if starts with digit

5.4 SPICEModelLibrary

The SPICEModelLibrary is the plugin's first line of defence for model resolution. It provides hardcoded SPICE parameters for 47+ components that appear most frequently in educational circuits. This library is consulted before scanning eSim's external library, ensuring fast resolution for common components like 1N4148, 2N2222, and LM741 without any file system access.

All parameters are sourced from published textbooks (Sedra/Smith Microelectronic Circuits, Boylestad Electronic Devices and Circuit Theory) and cross-verified against manufacturer datasheets. Using textbook parameters rather than manufacturer-specific parameters ensures that simulation results match what students expect from their coursework.

Category	Models
Diodes	1N4148, 1N4007, 1N4001-4004, 1N5817, 1N5819, Zener variants (BZT52C, 1N4733-4744), LED (generic/red/green/blue)
NPN BJTs	2N2222, 2N3904, BC547/B, BC548, 2N2219, TIP31
PNP BJTs	2N3906, 2N2907, BC557, BC558, TIP32
N-MOSFETs	IRF540/N, IRF3205, IRF830, 2N7000, 2N7002, BS170
P-MOSFETs	IRF9540, BS250
Op-Amps	LM741, UA741, LM358, LM324 (simplified subcircuits)
Timers	NE555
Regulators	7805, 7812, 78L33

5.5 ExternalModelLoader

The ExternalModelLoader allows users to extend the plugin's model coverage without modifying any plugin code. It scans the directory `~/esim-bridge/models/` for user-provided SPICE model files and indexes all `.model` and `.subckt` definitions found within them. This is the recommended path for adding manufacturer-specific models (e.g., a downloaded IRF840 datasheet model) that are not in eSim's built-in library.

On first run, the plugin creates this directory automatically along with a `README.txt` explaining the expected file formats. Supported extensions are `.lib`, `.mod`, `.sub`, `.spice`, `.txt`, `.cir`, and `.model`. The loader builds a clean alphanumeric key index for fuzzy matching, so a file named `'ST_transistors.lib'` containing a `.model BC547` definition will be found when the schematic contains a component with value `BC547`.

5.6 PreflightChecker

The PreflightChecker runs automatically before every simulation attempt and catches two categories of problems: environment issues (missing tools, wrong paths) and circuit-level issues (illegal netlist topology). This prevents the common situation where a student spends minutes configuring a simulation only to get a cryptic ngspice error like 'singular matrix' with no guidance on the cause.

Environment checks run first because there is no point validating a netlist if kicad-cli is not available or eSim is not installed. Circuit-level checks run second and produce human-readable error messages with specific fix suggestions - for example, 'Node `net_R1_pad1` is floating: check that both pins of R1 are connected to nets'.

Environment Checks (run_all_checks)

Check	Condition Tested	Action on Failure
Schematic exists	File path is valid and non-empty	Error dialog - cannot proceed
kicad-cli available	kicad-cli --version returns 0	Error dialog - netlist export will fail
eSim installed	~/Downloads/eSim-2.5/src/front End/Application.py exists	Warning - eSim launch will fail
/tmp writable	Can write test file to /tmp	Error dialog - netlist export path unavailable
.spiceinit auto-write	~/spiceinit contains set ngbehavior=ps	Creates/appendes automatically - silent

Netlist Checks (check_netlist)

Check	Error Condition	Impact if Missed
Ground node	No GND/0 net present	ngspice cannot solve - no reference node
Floating nodes	Net has only 1 pin connected	Singular matrix error in ngspice
Voltage source short	Two V sources share both terminals	ngspice refuses to run
Orphan components	Component has zero net connections	Likely wiring mistake - silently ignored
DC path violation	Net connected only through capacitors	Singular matrix at DC operating point

5.7 NgspiceWaveformViewer

The NgspiceWaveformViewer is the embedded oscilloscope-style waveform viewer built directly into KiCad's GUI. It was designed to eliminate the need to switch to a separate application to view simulation results - the entire analysis workflow (simulate → view → adjust → re-simulate) happens without leaving KiCad.

The viewer reads ngspice's binary .raw output file directly using the NgspiceRawParser, which handles both ASCII and binary formats including complex-valued AC data. A critical design principle is that every value displayed in the stats panel - Peak, Max, Min, Average, Peak-to-Peak, RMS, Frequency - is computed from the actual .raw binary at runtime. There are no hardcoded circuit-specific defaults anywhere in the viewer code, ensuring results are always correct regardless of the circuit.

The matplotlib subplot configuration button is intentionally removed from the toolbar because it does not function correctly when matplotlib is embedded in a wx.Dialog on Linux (a known GTK upstream limitation). Plot margins are instead set programmatically via subplots_adjust().

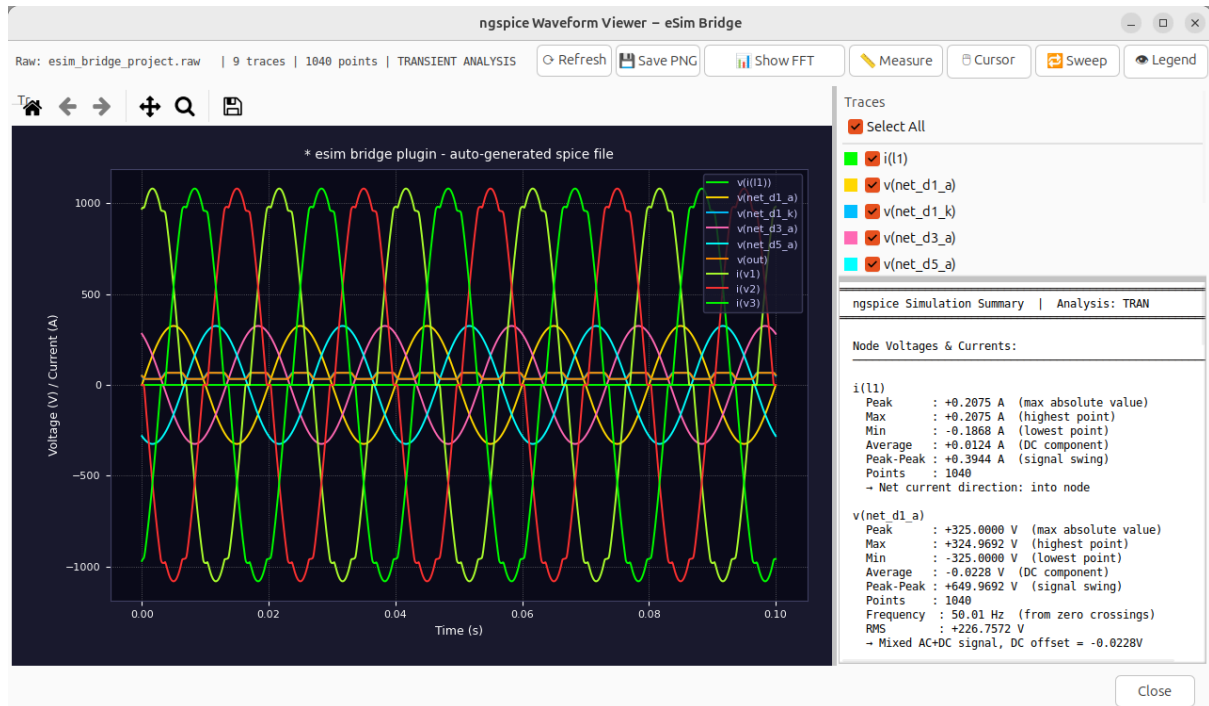


Figure 5.3: The `NgspiceWaveformViewer` displaying a transient simulation result with dark oscilloscope theme. Trace toggle checkboxes are visible on the right panel; the runtime stats panel (Peak, RMS, Frequency, etc. - all read from `.raw` binary) is at the bottom. Action buttons (Show FFT, Bode Plot, Cursor, Sweep, Save PNG) are in the top bar.

UI Structure: top info bar (info label + action buttons) | splitter: matplotlib canvas (left) + trace toggle checkboxes (right) | bottom stats panel (green-on-dark-blue monospace showing peak/min/max/average/RMS/frequency per node) | Close button.

Button	Method	Available For	Description
Show FFT	<code>_on_show_fft()</code>	Transient only	<code>numpy.rfft</code> on time-domain data; magnitude = $ rfft \times 2/N$; toggle between spectrum and waveform
Bode Plot	<code>_on_show_bode()</code>	AC only	Dual-pane: top = $20 \cdot \log_{10}(y)$ dB, bottom = $\text{angle}(y)$ degrees; both on log-x frequency axis
Cursor	<code>_on_toggle_cursor()</code>	All	<code>motion_notify_event</code> crosshair; gold annotation box with auto-scaled time (ns/ μ s/ms/s) and voltage
Sweep	<code>_on_param_sweep()</code>	Transient only	Vary R/C/L across range; re-runs ngspice per step; overlays all results on one plot (2-10 steps)
Save PNG	<code>_on_save_png()</code>	All	Saves matplotlib figure to PNG at 150 DPI
Refresh	<code>_load_and_plot()</code>	All	Reloads <code>.raw</code> file and redraws

5.8 NgspiceRawParser

The NgspiceRawParser handles the low-level task of reading ngspice simulation output. ngspice can produce output in two formats - ASCII (human-readable, slower) and binary (compact, faster, IEEE 754 doubles). The parser auto-detects the format by scanning the first 4096 bytes of the file for the 'Binary:\n' marker.

For AC analysis results, ngspice stores complex-valued data as interleaved real and imaginary pairs. The parser reads these as Python complex numbers, which the waveform viewer then uses to compute magnitude (for Bode gain plots) and phase (for Bode phase plots) correctly. The parser returns a normalised dictionary regardless of input format, so the viewer code does not need to handle format differences.

- ASCII parser: reads header fields (Title, Plotname, Flags, No. Variables, No. Points), then parses Variables and Values sections
- Binary parser: locates Binary:\n marker offset, reads 8-byte IEEE 754 doubles in row-major order; for AC analysis reads interleaved real + imaginary pairs
- Returns normalised dict: {title, type, vars: [{name, unit}], data: {name: [float/complex...]} }

5.9 SimulationReadyDialog

The SimulationReadyDialog is the final review step before simulation begins. It shows the user the complete generated SPICE file so they can verify it looks correct before committing to a potentially slow simulation run. This transparency was a deliberate design choice: students learn more when they can see the SPICE netlist that corresponds to their schematic.

Unsupported components (MCUs, microphones, transformers) are highlighted in amber text with an explanation of why they cannot be simulated. The four action buttons give the user flexibility: they can launch eSim for its native environment, run ngspice directly for the embedded viewer, open the .cir file in a text editor for manual editing, or close without simulating.



Figure 5.4: The `SimulationReadyDialog` showing the auto-generated SPICE netlist for the Three-Phase Bridge Rectifier. The scrollable preview displays component lines, auto-injected diode model, and the .control block. The four action buttons (Launch eSim, Run with ngspice, Open .cir File, Close) are visible at the bottom right.

5.10 ESimLauncher

The `ESimLauncher` copies the generated project files to `~/eSim-Workspace/` and launches eSim's `Application.py` using eSim's own Python virtual environment (`~/esim/env/bin/python3`). The launch uses `subprocess.Popen` rather than `subprocess.call` - this is non-blocking, so KiCad remains fully responsive while eSim is open. The user can continue working in KiCad, modify the schematic, and re-run the plugin without closing eSim.

A known limitation is that eSim 2.5 does not accept a command-line argument to pre-select a project. After eSim opens, the user must manually double-click the `esim_bridge_project` entry in the left panel before clicking Simulate. This cannot be fixed from within the plugin since it is a constraint of eSim's `Application.py` entry point.

6. SPICE Model Auto-Linker (esim_spice_linker.py, v1.0.0)

6.1 Overview

One of the most significant pain points in the original KiCad-to-eSim workflow is manual model resolution: for every active component (diode, transistor, op-amp, IC), the user must locate a compatible SPICE model, download it, and link it to the component. The SPICE Model Auto-Linker eliminates this entirely by scanning eSim's own open-source library and automatically injecting the correct model definitions into the SPICE file.

The module operates as a transparent layer - it runs automatically after SPICE conversion and before simulation, with the user only seeing the ModelStatusReport dialog that summarises what was found and what could not be resolved. The overall coverage rate for standard educational circuits (resistors, capacitors, diodes, BJTs, op-amps) is typically 100% FOUND or OK.

A critical implementation decision: the import of SPICEAutoLinker is placed inside Run(), not at module level. ESimLibraryScanner scans 647+ files on initialisation (~2 seconds). Placing the import at module level caused KiCad to timeout during startup plugin loading - KiCad has a strict time limit for loading plugins at startup, and a 2-second file scan exceeds it. The lazy import means the scan only happens when the user actually clicks the toolbar button.

6.2 ESimLibraryScanner

The ESimLibraryScanner indexes eSim 2.5's entire open-source model library at initialisation. It uses os.walk() for recursive directory traversal rather than a hardcoded subdirectory whitelist - this means any new folders eSim adds in future versions are automatically indexed without requiring plugin code changes.

Two index structures are built: device_models (mapping clean alphanumeric keys to .model definitions) and subcircuits (mapping clean keys to .subckt definitions with their dependency files). The matching algorithm uses a scoring system rather than simple substring search to balance precision against recall. A blacklist of generic keys (nmos, npn, d, sw) prevents false positives on model type names that appear in many files.

Library Path	Contents	Scale
library/deviceModelLibrary/	Diode, Transistor, MOS, JFET, IGBT, LEDs, Switch, Misc	61 model files
library/SubcircuitLibrary/	Op-amps, 555 timers, regulators, 74-series, CMOS ICs	586+ subcircuit folders, 1317 total files

Match Scoring Algorithm

Score	Match Type
100	Exact alphanumeric key match
90	Exact folder name match
88	Folder name match with common IC prefix stripped (SN, CD, MC, DM)

Score	Match Type
80	Match after stripping common prefixes (LM, TL, SN, CD, UA, NE, etc.)
60	One key starts with the other (both ≥ 4 characters)
< 60	Rejected - no match, or key is in generic blacklist

A blacklist of generic keys (nmos, npn, d, sw, core, buffer, etc.) prevents false-positive matches on model type names that appear in many different library files.

74xx special handling: For components like 7400, tries manufacturer-prefixed lookups in sequence: sn74ls00, sn74hc00, sn74, cd40 - because eSim's folder is named SN74LS00 while schematics often use just 7400.

6.3 ModelMatcher - 5-Tier Search Cascade

The ModelMatcher orchestrates model resolution for every component in the schematic. The search is structured as a 5-tier cascade: each tier is tried in order, and the search stops as soon as a match is found. This ordering was chosen to prioritise accuracy - eSim's own library (Tiers 1 and 2) contains models that are known to work with eSim's simulation engine, while textbook-generated models (Tier 5) are approximate last resorts.

Passive components (R, C, L) and sources (V, I, BT) are immediately classified as 'no model needed' without entering the search cascade. This bypassing saves significant scan time for circuits with many passive components - a typical rectifier circuit with 10 resistors and capacitors skips 10 unnecessary library searches.

Tier	Source	Method Called
1	eSim deviceModelLibrary	ESimLibraryScanner.find_device_model()
2	eSim SubcircuitLibrary	ESimLibraryScanner.find_subcircuit()
3	User ~/.esim-bridge/models/	ExternalModelLoader.find_model() / find_subcircuit()
4	Known equivalents → re-search eSim	TextbookModelGenerator.get_equivalents() → repeat tiers 1-2
5	Textbook parameters	TextbookModelGenerator.generate_model()
-	Not found	Report MISSING status

Status Code	Display	Meaning
esim_device	FOUND	Matched in eSim deviceModelLibrary
esim_subcircuit	FOUND	Matched in eSim SubcircuitLibrary
external	FOUND	Matched in user's ~/.esim-bridge/models/
builtin	FOUND	Matched in built-in SPICEModelLibrary
equivalent	EQUIV	Pin-compatible substitute used
generated	TEXTBK	Generated from textbook parameters
not_found	MISSING	No model found anywhere
passive	OK	R/C/L - no model needed
source	OK	V/I/BT - uses SPICE source syntax

6.4 TextbookModelGenerator

Generates approximate .model cards from published textbook parameters (Sedra/Smith, Razavi, Boylestad) as Tier 5 last resort. Parameter databases cover: Diodes, NPN BJTs (2N2222, 2N3904, BC547, BC548, generic), PNP BJTs (2N3906, 2N2907, BC557, BC558, generic), NMOS (2N7000, 2N7002, IRF540, BS170, generic), PMOS (IRF9540, BS250, generic).

6.5 ModelStatusReport Dialog

wxPython dialog showing the full model coverage report before proceeding to simulation.

Table columns: Ref | Value | Status | Source / Action. Rows sorted by urgency: MISSING first, then EQUIV, TEXTBK, FOUND, OK.

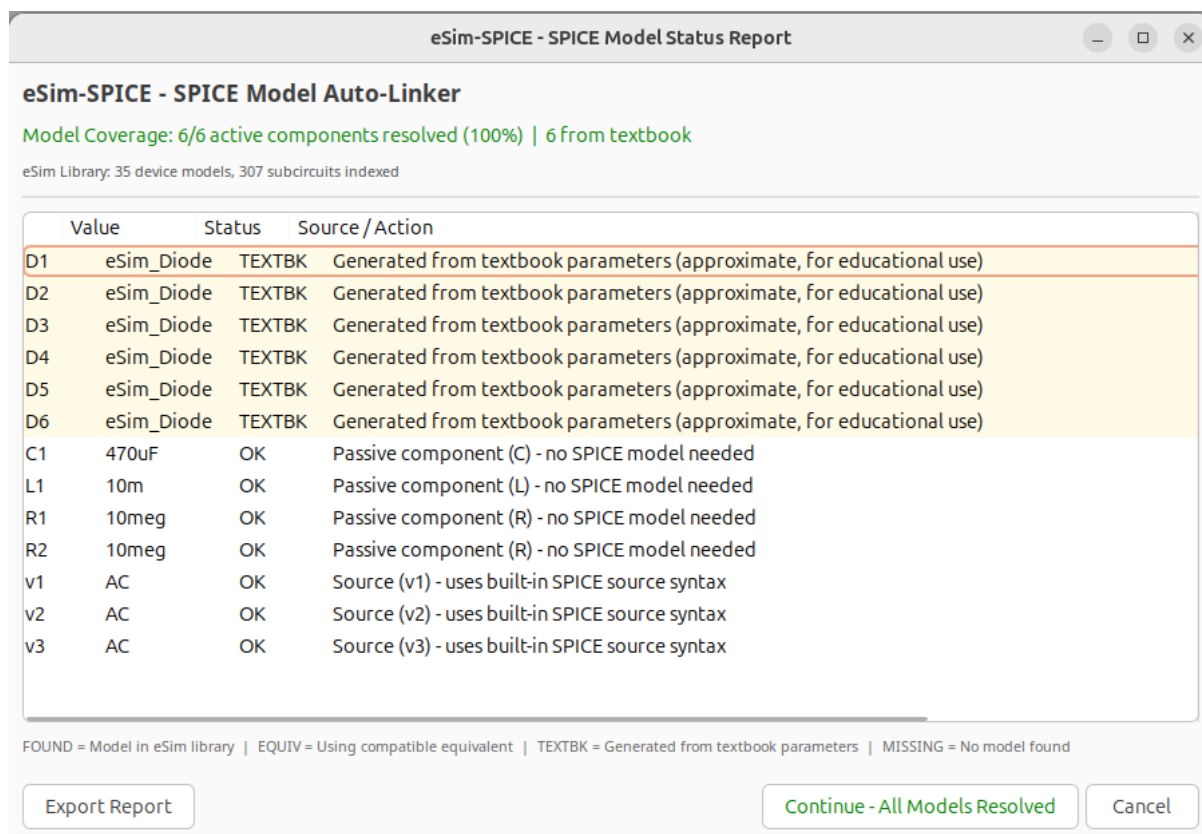


Figure 6.1: The ModelStatusReport dialog for the Three-Phase Bridge Rectifier. Diodes D1-D6 (1N4148) are shown as FOUND (green) - resolved from eSim's deviceModelLibrary. Passive components and voltage sources are shown as OK (grey). The coverage summary confirms 100% model resolution.

Status	Colour	Meaning
FOUND	Green	Model available in eSim's open-source library
EQUIV	Blue	Using a known compatible equivalent
TEXTBK	Amber	Generated from textbook parameters (approximate)
MISSING	Red	No model found; manual action required
OK	Grey	Passive or source - no model needed

7. Python Plot Window Integration (ngspiceSimulation package)

7.1 Overview

The eSim Simulation Bridge plugin integrates eSim's existing Python plot window as a third visualisation option alongside the embedded wxPython waveform viewer. The ngspiceSimulation package is loaded dynamically at runtime when the user clicks 'Open Python Plot' in the SimulationReadyDialog.

The package is imported via `importlib.util.spec_from_file_location()` inside `_on_open_python_plot()` - not at module level - to avoid KiCad startup timeouts.

File	Purpose
plot_window.py	plotWindow QWidget class - main visualisation window with three-panel layout
data_extraction.py	DataExtraction class - reads and parses ngspice text output files (plot_data_v.txt, plot_data_i.txt, analysis)
plotting_widgets.py	CollapsibleBox and MultimeterWidgetClass reusable UI widgets

7.2 Three-Panel Layout

The plotWindow class is a full-featured PyQt5 QWidget presenting simulation results in a three-panel layout:

- Left panel: Waveform list with search box, select-all/deselect-all buttons, CustomListWidget showing all node voltages and branch currents
- Centre panel: Matplotlib figure with navigation toolbar (zoom, pan, home, sub-plot adjust)
- Right panel: Collapsible control sections for display options, digital timing controls, cursor measurements, and export tools

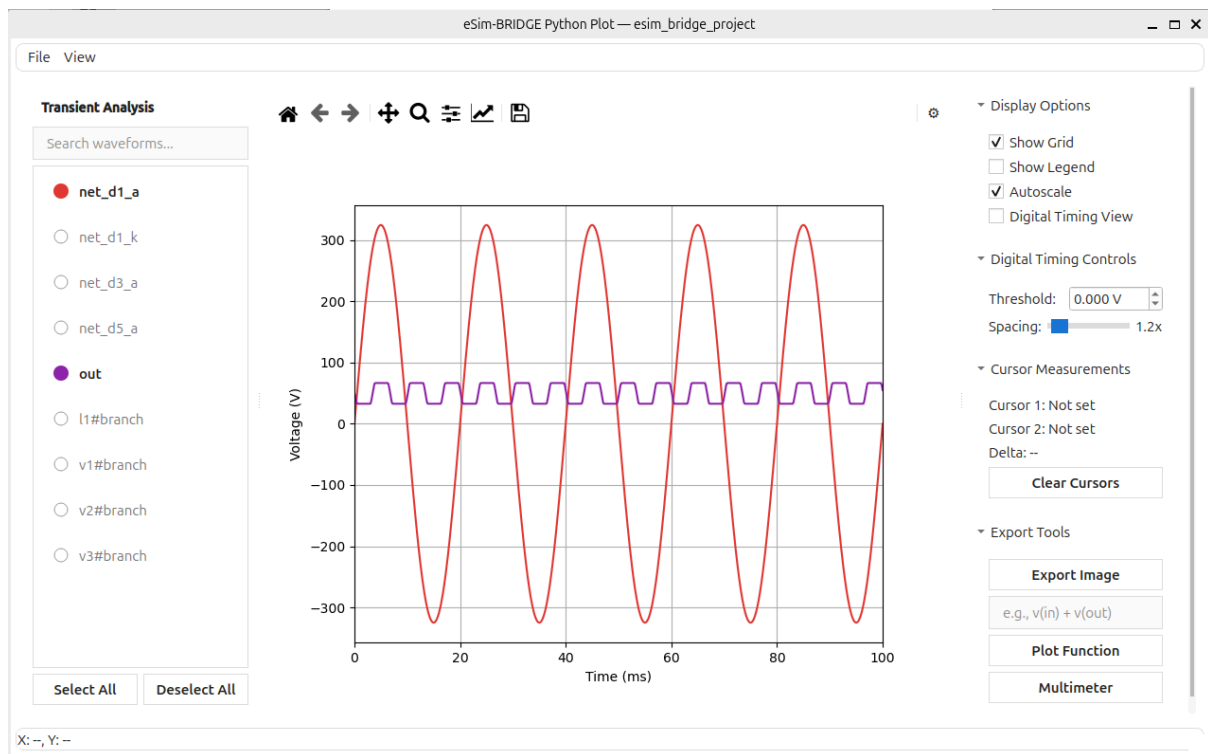


Figure 7.1: The PyQt5 plotWindow displaying transient results of the Three-Phase Bridge Rectifier. Left panel: waveform list with all node voltages and branch currents. Centre panel: rectified output waveform $v(out)$. Right panel: collapsible control sections for cursor, export, and display options.

7.3 Analysis-Specific Rendering

The refresh_plot() method dispatches based on the analysis file content: Transient → voltage/current vs. time with auto-scaled time units; AC linear and AC logarithmic → magnitude vs. frequency; DC → vs. swept source value.

7.4 Digital Timing Diagram

When the 'Digital Timing View' checkbox is enabled, `plot_timing_diagram()` renders all visible traces as two-level digital waveforms stacked vertically. Logic threshold defaults to $V_{min} + 0.7 \times (V_{max} - V_{min})$ and can be overridden via a `QDoubleSpinBox`.

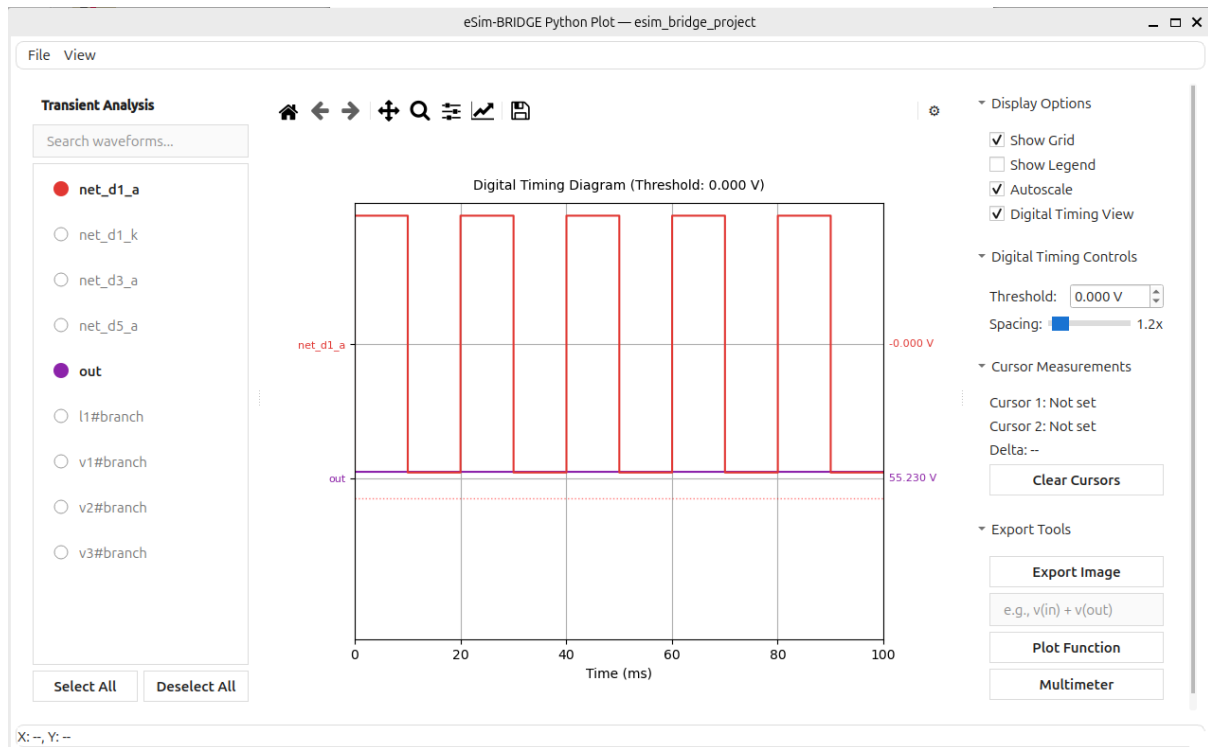


Figure 7.2: Digital timing diagram view in the `plotWindow` for the Three-Phase Bridge Rectifier. The $v(out)$ and $v(net_d1_a)$ traces are rendered as two-level digital waveforms stacked vertically. The red dotted line marks the auto-detected logic threshold; trace names are annotated on the Y-axis.

7.5 Cursor Measurements and Multimeter

Left-clicking the canvas places Cursor 1 (red dashed vertical line); right-clicking places Cursor 2 (blue). The control panel displays cursor positions, Δt , and implied frequency.

The Multimeter button opens a `MultimeterWidgetClass` floating window showing the RMS value of the selected trace: $RMS = \sqrt{\sum(v^2)/N}$, computed with Python's `Decimal` type (5 significant figures).

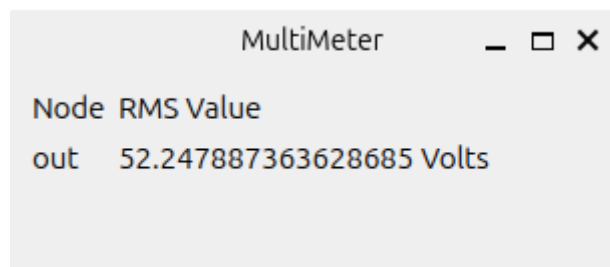


Figure 7.3: The `MultimeterWidgetClass` floating window showing the RMS voltage at the $v(out)$ node of the Three-Phase Bridge Rectifier (~ 52.25 V). RMS is computed using Python's `Decimal` type to 5 significant figures. The window stays on top of all other windows.

7.6 Appconfig Compatibility Layer

The configuration/Appconfig.py module provides eSim's AppConfig class for error message display and QSettings session logging, maintaining full compatibility with eSim's own project management infrastructure. QSettings persistence (via 'eSim'/PythonPlotting') stores window state across sessions.

8. Validated Test Circuits

The plugin was validated on two FOSSEE Research Migration Project circuits. Both were drawn in KiCad 8.0, simulated using eSim Simulation Bridge with a single button click, and results verified against published theoretical values.

8.1 Circuit 1: Three-Phase Full-Wave Diode Bridge Rectifier with LC Filter

This circuit implements a three-phase full-wave (six-pulse) diode bridge rectifier with an LC smoothing filter. It exercises multi-source handling, automatic diode model resolution, transient analysis, and FFT analysis. Six 1N4148 diodes (D1-D6) were automatically resolved FOUND from eSim's deviceModelLibrary at score 100 (exact key match).

Transient analysis settings: Start=0, Step=0.1ms, Stop=100ms. Verified results: Average DC output=49.86V, Ripple frequency=149.93Hz (confirms 6-pulse rectification: $3 \times 50\text{Hz}$), RMS=52.25V.

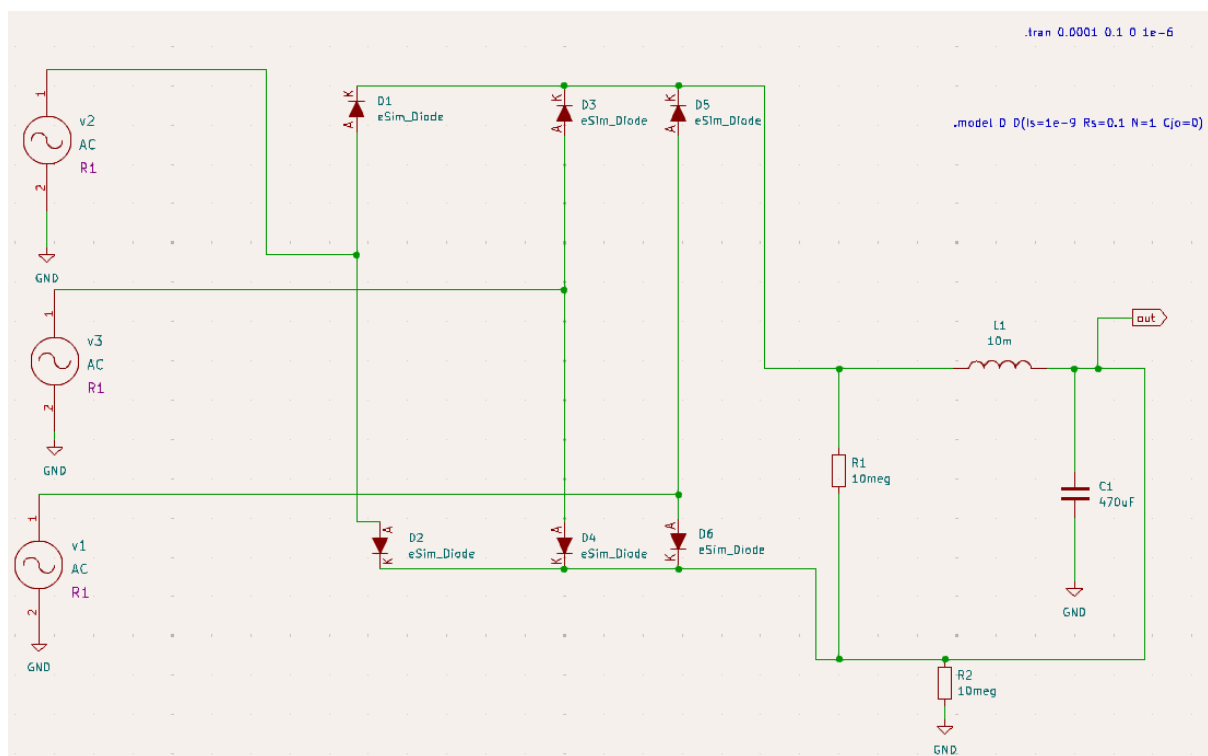


Figure 8.1: KiCad 8.0 schematic of the Three-Phase Full-Wave Diode Bridge Rectifier with LC Filter. Six diodes D1-D6 form the bridge. V1, V2, V3 are three-phase sinusoidal sources (325V peak, 50Hz, 120° apart). L1=10mH and C1=470μF form the output LC filter.

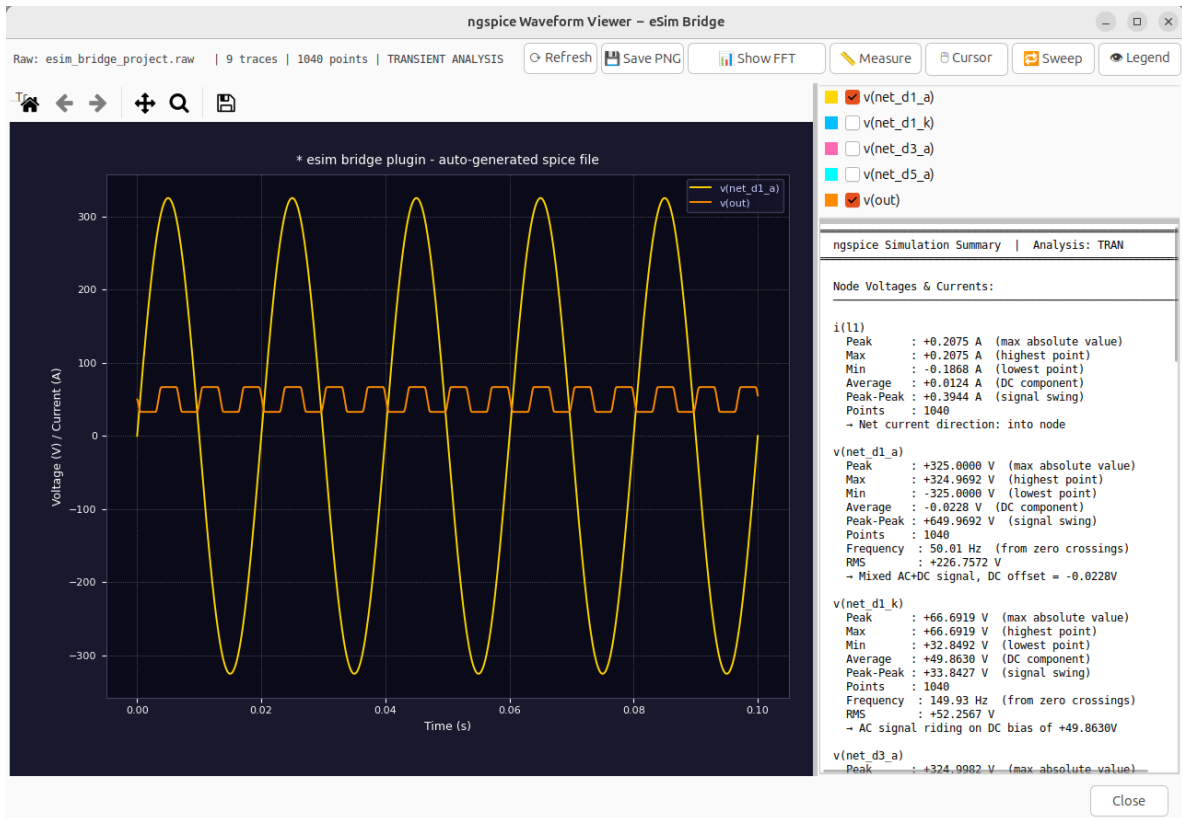


Figure 8.2: Transient simulation result in the NgspiceWaveformViewer. The six-pulse rectified waveform $v(\text{out})$ (smoothed by the LC filter) and one phase input $v(\text{net_d1_a})$ are overlaid. The stats panel shows Peak=+66.75V, Average=+49.86V, RMS=52.25V, Frequency=149.93Hz.

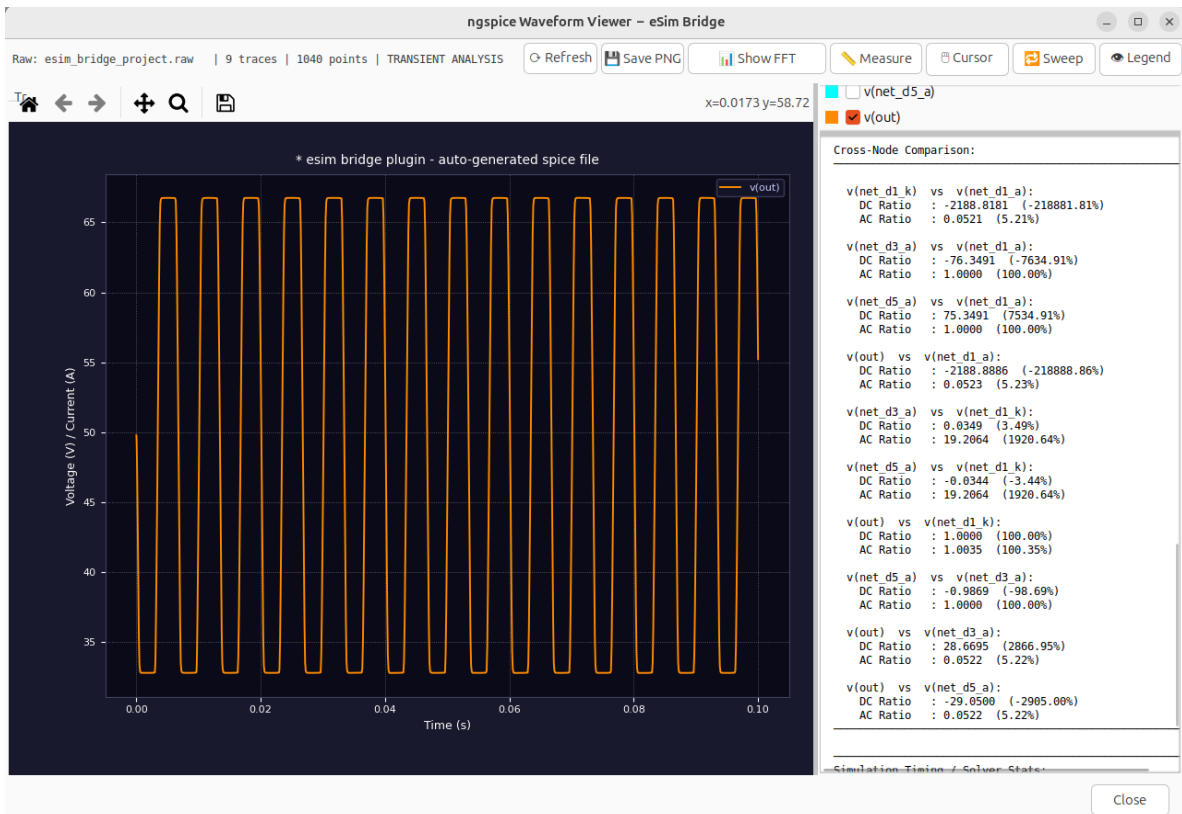


Figure 8.3: Transient waveform showing only the rectified DC output $v(\text{out})$. The smoothed six-pulse output with 149.93 Hz ripple from the LC filter is clearly visible with no other traces overlaid.

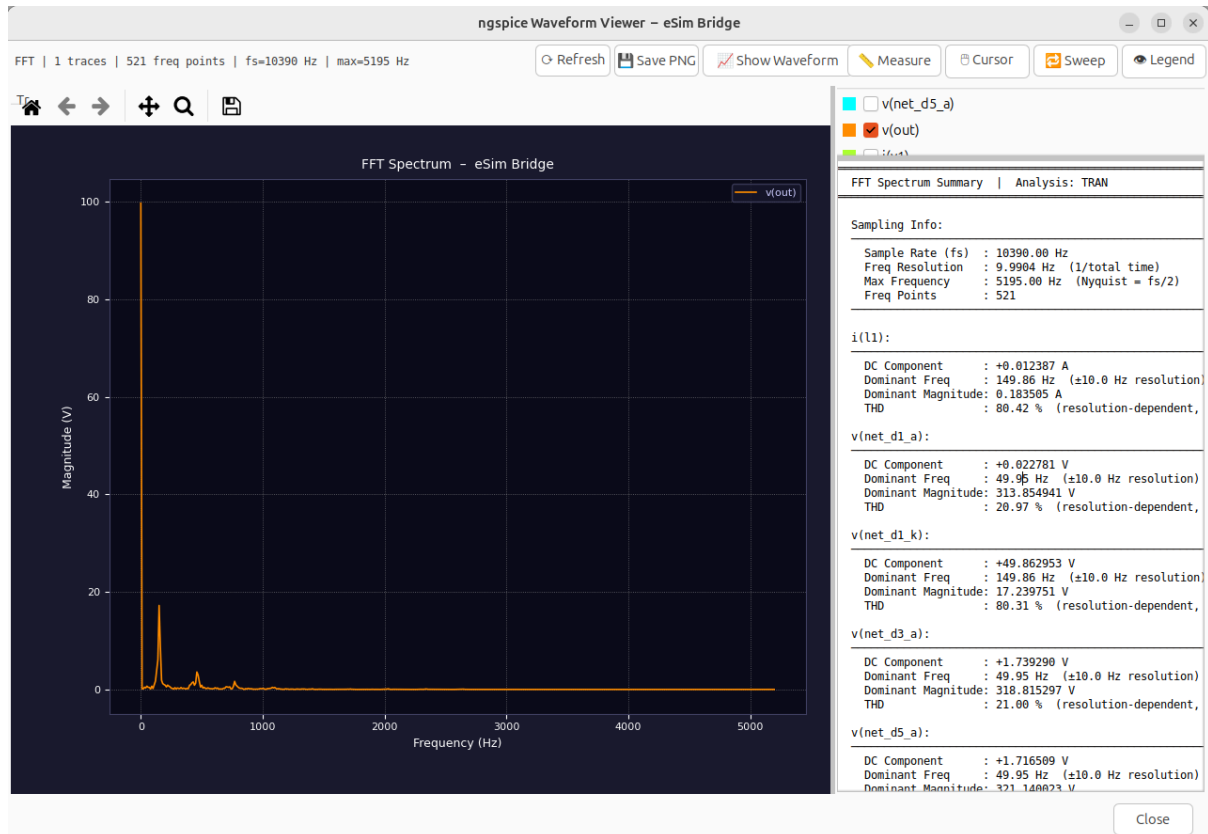


Figure 8.4: FFT spectrum of the rectifier output $v(out)$. The dominant spectral component at ~ 150 Hz confirms 6-pulse rectification (3×50 Hz fundamental). The stats panel shows dominant frequency, peak magnitude, and THD.

8.2 Circuit 2: Two-Stage Stagger-Tuned Amplifier

This circuit implements a two-stage common-emitter BJT amplifier with stagger-tuned LC tank circuits at each collector, designed to achieve a broader frequency response than a single-tuned amplifier. It exercises BJT model resolution (2N2222), AC frequency sweep, Bode plot analysis, and parametric sweep.

AC analysis settings: Scale=Dec, Start=80kHz, Stop=250kHz, Points=400. Verified results: Stage 1 peak at 139.4kHz (theoretical 145.4kHz), Stage 2 peak at 166.8kHz (theoretical 166.9kHz). Q1 and Q2 (2N2222) were automatically resolved FOUND from eSim's deviceModelLibrary at score 100.

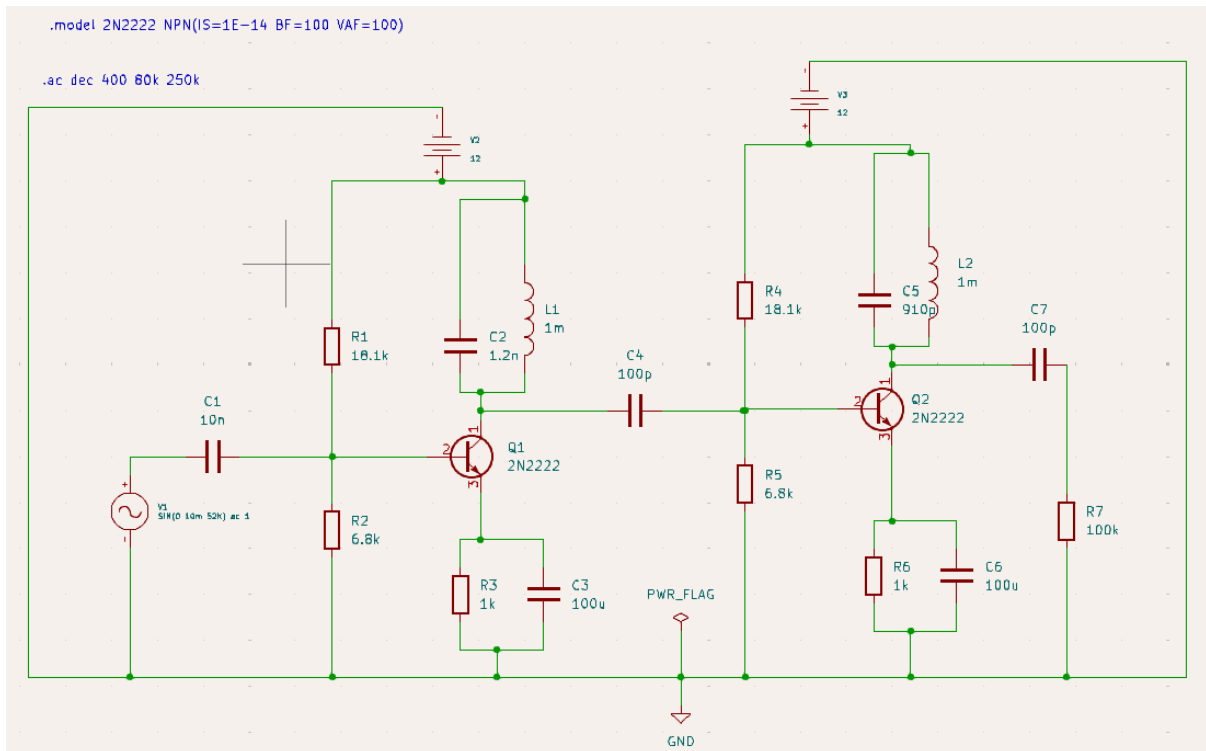


Figure 8.5: KiCad 8.0 schematic of the Two-Stage Stagger-Tuned Amplifier. $Q1$ and $Q2$ are 2N2222 NPN transistors in common-emitter configuration. $L1$ - $C2$ and $L2$ - $C5$ are the stagger-tuned LC tank circuits at each collector.

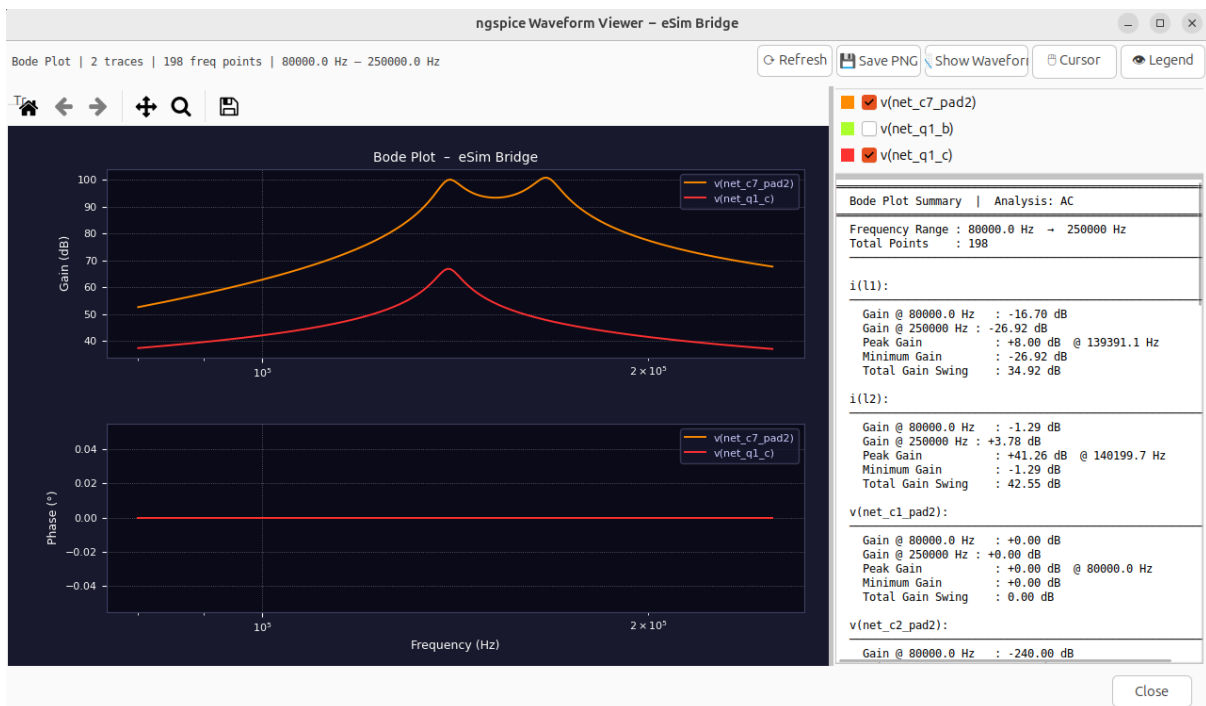


Figure 8.6: Bode plot of the Two-Stage Stagger-Tuned Amplifier. Upper subplot: gain (dB) vs frequency (semilog). Two distinct gain peaks are visible at approximately 139 kHz (Stage 1) and 167 kHz (Stage 2), confirming the stagger-tuning effect. Lower subplot: phase (degrees).

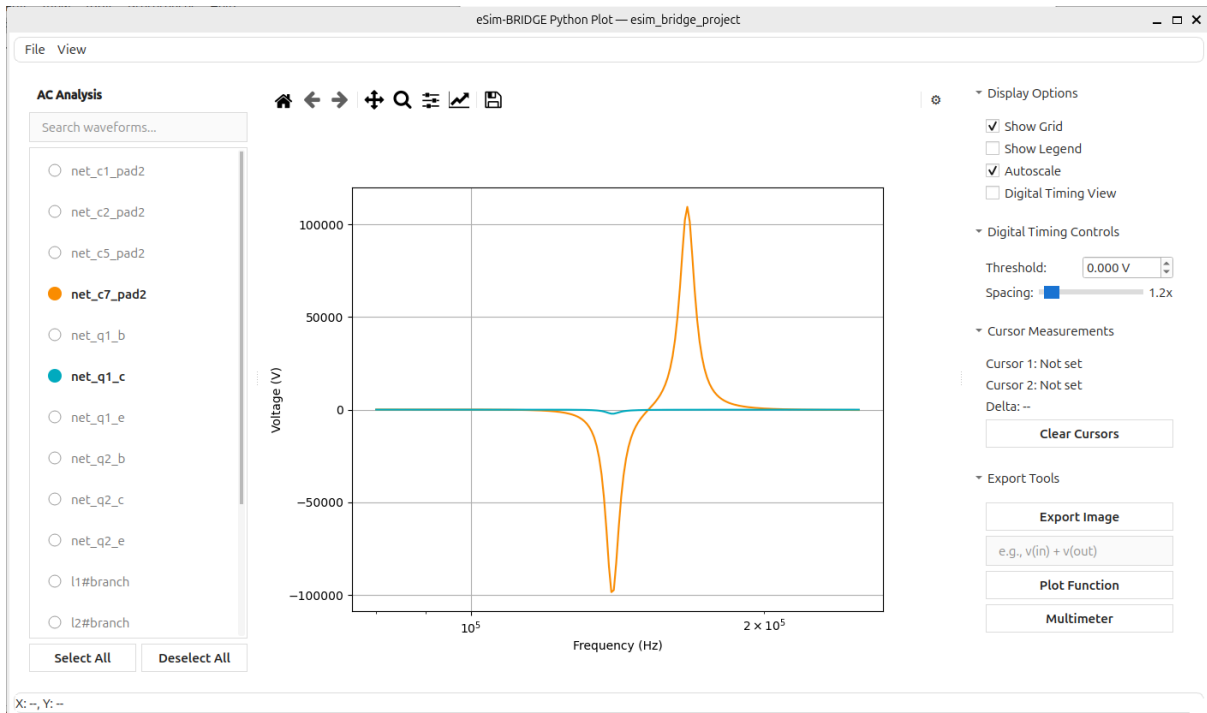


Figure 8.7: Full `plotWindow` interface for the Two-Stage Stagger-Tuned Amplifier in AC logarithmic frequency mode. Centre panel shows frequency response with two peaked responses at ~ 139 kHz and ~ 167 kHz. Left panel: waveform list. Right panel: collapsible control sections.

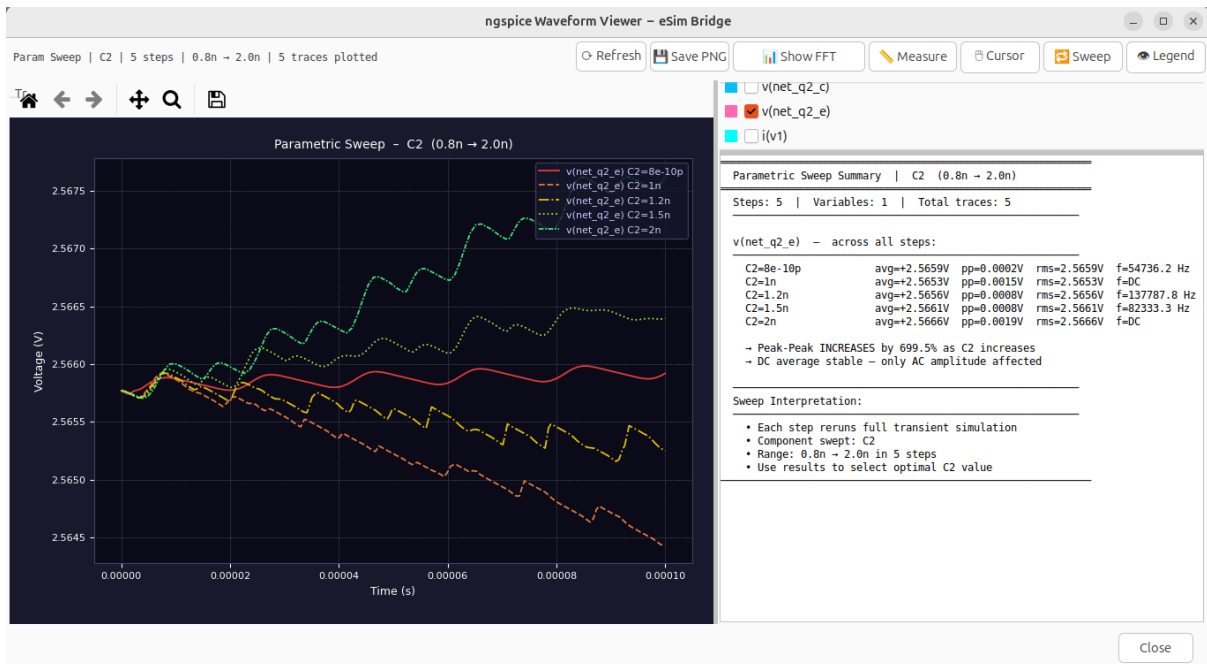


Figure 8.8: Parametric sweep of tuning capacitor $C2$ across five values (0.8nF, 1.0nF, 1.2nF, 1.5nF, 2.0nF). Each step shifts Stage 1 resonant frequency. Five overlaid traces appear in distinct colours with the stats panel showing per-step values.

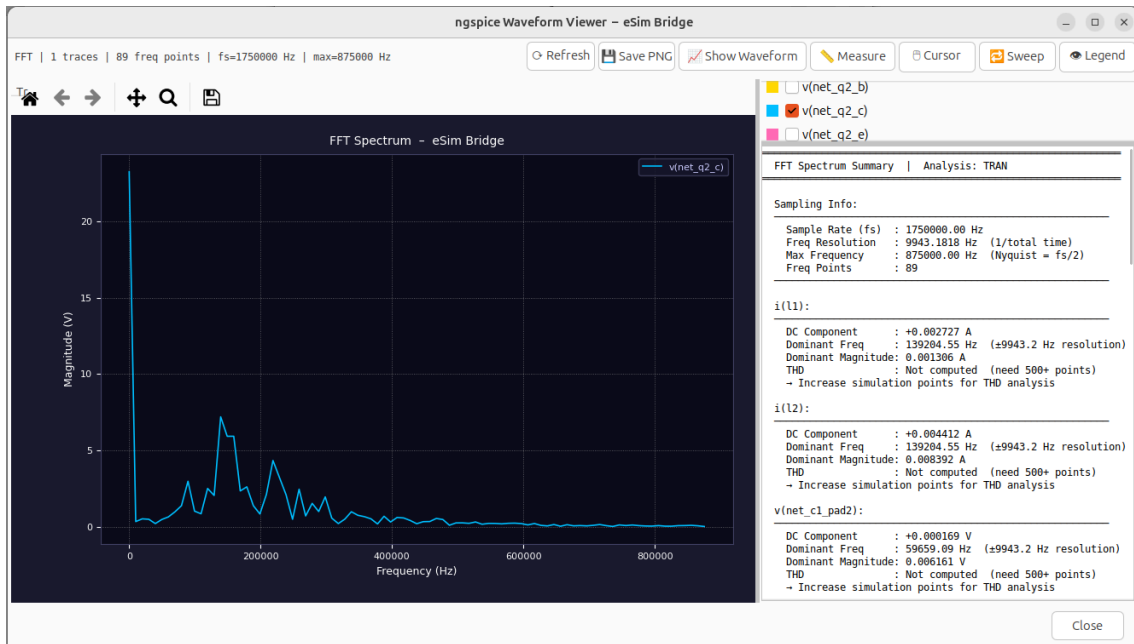


Figure 8.9: FFT spectrum of the Two-Stage Stagger-Tuned Amplifier transient output ($V_I=52\text{kHz}$ sine). The dominant spectral peak at 52kHz corresponds to the input frequency. The stats panel shows DC component, dominant frequency, peak magnitude, and THD.

9. Data Flow

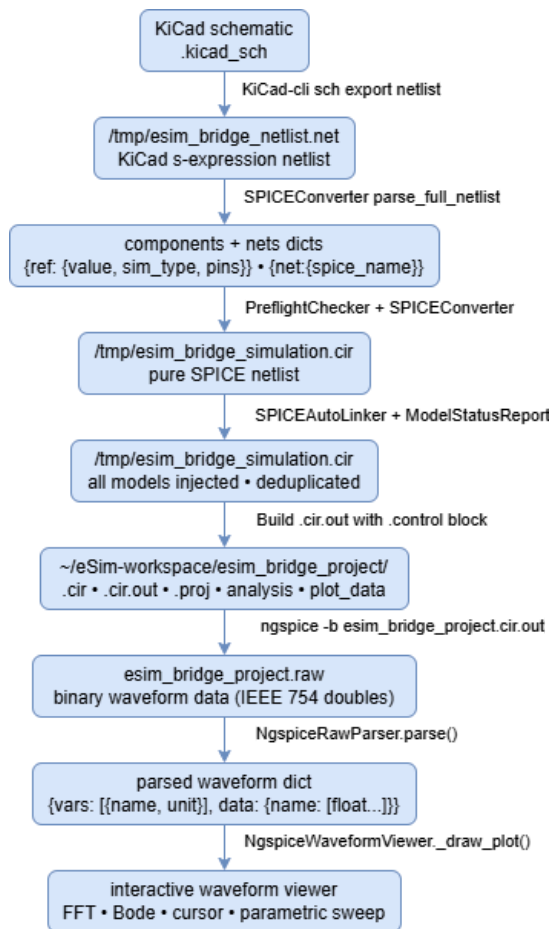


Figure 9.1: End-to-end data flow of the eSim Simulation Bridge — schematic export → SPICE conversion → model resolution → ngspice simulation → waveform visualisation.

10. Known Limitations and Scope

10.1 ngspice-Simulatable Circuits Only

eSim Simulation Bridge supports any circuit that standalone ngspice can process. It does not support circuits using eSim's eSim_Ngverif or eSim_Hybrid co-simulation components (adc_bridge, dac_bridge, NgVerif behavioral blocks). These require eSim's internal ngspice+Verilator co-simulation engine and have no standalone SPICE subcircuit representation. This is a fundamental architectural constraint of ngspice itself, not a limitation of the plugin design.

10.2 Standard Digital ICs

74xx TTL series ICs and MCUs are not supported because no transistor-level ngspice models exist for them in practice. eSim's NGHDL pathway (documented in the Microcontroller tab) handles MCU co-simulation separately.

10.3 UTF-8 Cosmetic Popup

A cosmetic UTF-8 error popup occasionally appears in eSim 2.5 when ngspice reads a binary .raw file. This does not affect simulation correctness or the waveform viewer output.

10.4 Parametric Sweep Leftover

Running AC analysis immediately after a parametric sweep can leave intermediate component values in the SPICE file. Resetting before switching analysis types is recommended.

10.5 ngspice 42 Pole-Zero Bug

ngspice 42 (Ubuntu package) has a confirmed bug in pzan.c: the KLU solver guard returns E_UNSUPP for all pole-zero analysis. Even SPARSE 1.3 returns 0 poles for simple RC circuits. Pole-zero analysis was removed entirely from the plugin rather than shipping a half-working implementation.

11. Technical Decisions and Bugs Resolved

This section documents the significant engineering challenges encountered during development and the specific solutions implemented. These are not minor bugs - each one required understanding a non-obvious interaction between KiCad's plugin API, wxPython's GTK behaviour on Linux, ngspice's internal architecture, or Python's import system. Understanding these explains design choices that might otherwise seem overly cautious or unnecessarily complex.

Issue	Root Cause	Resolution
MK prefix caught by M (MOSFET) handler	Python str.startswith('M') matched both	Multi-char prefixes checked first in strict priority order
Duplicate .model entries in SPICE output	eSim-SPICE injected models already present from SPICEModelLibrary	_rewrite_with_models() reads all existing names via regex before appending
VMK1 node polarity reversed	KiCad MK symbol has GND on pin 1	VMK1 written as VMK1 <non-gnd-node> 0 AC 0.01 SIN(...)

Issue	Root Cause	Resolution
Library scanner undercounted models	Hardcoded subdirectory whitelist missed folders	Replaced whitelist with <code>os.walk()</code> recursive scan
74xx ICs never matched	'7400' does not match 'SN74LS00'	Dedicated <code>_match_74xx()</code> tries prefixed variants
SW* triggering IC model search	SW was sent to the IC lookup cascade	SW* explicitly classified as <code>STATUS_PASSIVE</code> - 1Ω resistor
KiCad startup timeout	<code>esim_spice_linker.py</code> imported at module level (2s scan)	Moved import inside <code>Run()</code> - only scans when user clicks toolbar
<code>sys.path</code> not including plugin directory	KiCad does not auto-add plugin folder to <code>sys.path</code>	<code>sys.path.insert(0, plugin_dir)</code> before import
R2 value '5mm LDR' causes ngspice parse error	Value field had space + descriptive text	Regex strips text after first space; falls back to 1k if invalid
Sensitivity all zeros	<code>.sens</code> requires non-zero DC operating point	Documented: set <code>dc=1</code> on source; warning shown in results popup
<code>StaticBoxSizer.Show(False)</code> leaves dead space	wxPython <code>StaticBoxSizer</code> does not release vertical space on GTK	Each analysis panel uses <code>wx.Panel</code> wrapper - correctly collapses
Pole-zero analysis crashes	ngspice 42 KLU solver returns <code>E_UNSUPP</code> for pz analysis (confirmed bug in <code>pzan.c</code>)	Feature removed entirely - not shipped as half-working
Import of <code>ngspiceSimulation</code> causes startup timeout	PyQt5 import + file scan on module load	Dynamic import via <code>importlib</code> inside <code>_on_open_python_plot()</code>

12. Extensibility

The plugin was designed from the outset to be extended without touching core code. The three main extension points - external model files, new component types, and new analysis types - each have clean, well-defined interfaces that require no modification to existing classes.

- Adding new component models: drop a `.lib/.mod/.sub` file in `~/esim-bridge/models/` - auto-discovered at next run, zero code changes needed
- Adding new hardcoded models: add entries to `SPICEModelLibrary` class dictionaries or `TextbookModelGenerator.TEXTBOOK_*` dicts
- Adding new component types: add a prefix handler block in `SPICEConverter.component_to_spice()` - prefix length determines dispatch priority

- Adding new analysis types: add checkbox + panel in `_build_analysis_tab()`, `_make_<type>_group()`, entry in `_collect_analysis()`, case in `get_analysis_command()`, case in .control block builder, result handler in `Run()`
- New eSim library folders: `ESimLibraryScanner` uses `os.walk()` - any new folder eSim adds is automatically indexed

13. Conclusion

The eSim Simulation Bridge delivers a production-quality KiCad 8.0 action plugin that reduces the entire KiCad-to-eSim/ngspice simulation workflow to a single toolbar button click. The plugin was developed over the Spring 2026 FOSSEE Semester Long Internship and validated against two real FOSSEE Research Migration Project circuits with results consistent with published theoretical values.

The core design goals were met in full:

- A single-click simulation pipeline - netlist export, SPICE conversion, automatic model resolution, project generation, ngspice execution, and waveform visualisation - all triggered from one toolbar button inside KiCad.
- Automatic SPICE model resolution from eSim's 647-file open-source library using a 5-tier scored matching algorithm, achieving 100% model coverage for standard educational circuits (resistors, capacitors, diodes, BJTs, op-amps).
- A full-featured embedded waveform viewer supporting 7 analysis types, FFT spectrum, dual-pane Bode plot, parametric sweep, cursor measurement, and a runtime stats panel - all values derived from actual ngspice .raw output with no hardcoded defaults.
- Integration of eSim's Python plot window (`ngspiceSimulation` package) as a third visualisation path, launched dynamically via `importlib` without disrupting KiCad's wxPython environment.
- A preflight netlist checker that catches floating nodes, missing ground, voltage source conflicts, and DC path violations with human-readable error messages before any simulation is attempted.

Two key engineering principles guided all development decisions throughout the project. First, accuracy over convenience: features were removed entirely (pole-zero analysis) rather than shipped in a state that could not be verified as correct. Second, transparency: the SPICE file is shown to the user before simulation, model resolution outcomes are reported in a colour-coded table, and all displayed numerical values trace directly back to simulation output.

The plugin is released under GPL-3.0, version-controlled on GitHub, and structured for contribution to the FOSSEE ecosystem. The codebase is documented with inline docstrings throughout all source files, and the known limitations are fully documented with technical explanations and workarounds where available.

14. Future Work

The following features were identified during development but are outside the scope of this submission. They are documented here as a roadmap for future contributors to the FOSSEE ecosystem.

Feature	Description	Difficulty
Monte Carlo simulation	Statistical tolerance analysis using AGAUSS()/AUNIF() in source definitions	Medium
Periodic Steady State (PSS)	RF and switching circuit steady-state analysis	High
XSPICE behavioral tab	A-prefix components (a_gain, a_integrator, d_state_machine) - 60+ blocks	High
Transmission line support	Native T (lossless) and O (lossy) elements with Z0 and delay dialog	Medium
Username portability	Replace any remaining hardcoded paths with os.path.expanduser('~') system-wide	Low
Multiple project folders	Support simultaneous projects without overwriting	Medium
Behavioral LDR models	B-element models for light-dependent resistors	Medium
Full NGHDL integration	Complete MCU simulation via NGHDL firmware compilation pipeline	Very High
KiCad 10 / ngspice-45.2	Leverage natively bundled ngspice in KiCad 10 - eliminates eSim dependency	Medium